ADC-ASP and AVG-ASP for a TDMA-MIN HMPSoC Frequency-Measurement Pipeline

COMPSYS 701 Advanced Digital Design, Individual Research Project 2026
Student scope: ADC-ASP and AVG-ASP under `submission/`
Target platform: Cyclone V 5CSEMA5F31C6, TDMA-MIN critical subsystem

1. Introduction and Scope

The Individual Research Project asks each student to design one or two Application-Specific Processors (ASPs) that can operate as nodes in the critical part of the Heterogeneous Multiprocessor System-on-Chip (HMPSoC). The course architecture separates non-critical software-oriented processing from a critical subsystem connected through a time-predictable TDMA-MIN Network-on-Chip (NoC). In the intended frequency-measurement case study, an analogue power-system voltage is sampled, filtered, processed by a reference-point or correlation algorithm, and then converted into frequency information by a processor such as Nios II.

My responsibility in the group pipeline is the input sampling ASP and the moving-average filter ASP. The first block, ADC-ASP, emulates an analogue-to-digital converter by generating deterministic signed sample streams. This allows the rest of the TDMA-MIN pipeline to be tested even when no physical ADC interface is connected. The second block, AVG-ASP, is a Data Processing ASP (DP-ASP) that filters signed samples with a configurable moving average. Together, these two blocks provide the front end of the frequency-measurement chain:

ADC-ASP -> AVG-ASP -> COR-ASP -> Peak Detection ASP -> Nios II / software

The two implemented RTL blocks use a shared 32-bit packet interface and ready/valid handshaking. They are written in VHDL-2008 and are currently located at:

- ` submission/adc_asp/adc_asp.vhd`
- ` submission/avg_asp/avg_asp.vhd`
- ` submission/common/asp_packet_pkg.vhd`

The design was verified in ModelSim using existing and new testbenches, then synthesized as standalone IP-style top levels in Quartus Prime 18.1 for the same Cyclone V device used by the group ReCOP project.

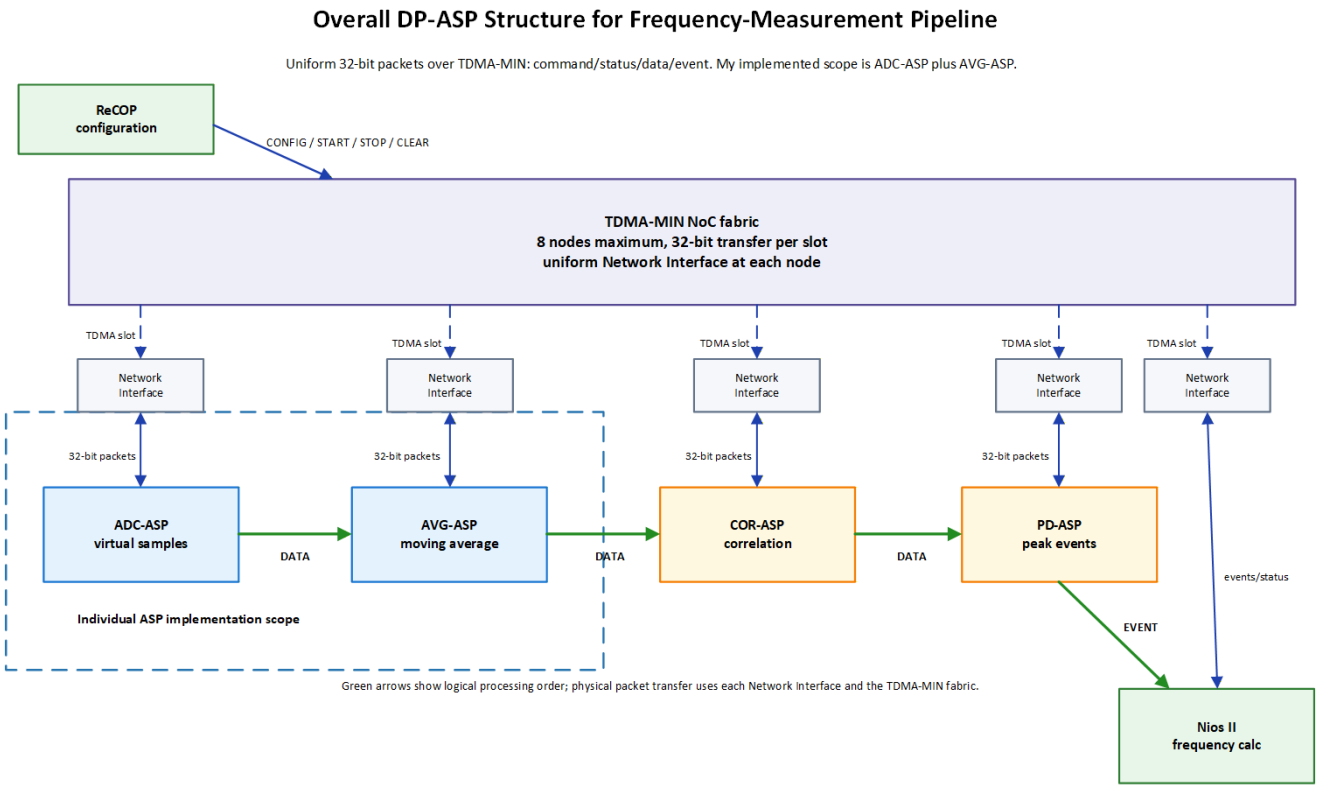

Figure 1. Overall DP-ASP pipeline structure.

2. Algorithm and Application Background

The project brief describes a power-system frequency-measurement application based on reference-point detection. The input is a periodic power signal, normally around 50 Hz, sampled at a fixed sampling rate. The supplied power-signal model notes specify a 16 kHz sampling rate and discuss storing several cycles of the modelled signal in FPGA memory. The reference-point detection notes explain the processing chain: ADC sampling, moving-average filtering, correlation or symmetry-related processing, peak detection, and finally frequency calculation from the number of samples between successive reference points.

The purpose of the ADC-ASP in this project is not to be a complete electrical ADC controller. Instead, it is a realistic digital source ASP that mimics sampled input data. It produces repeatable sample streams from virtual channels, which is useful for simulation, NoC routing tests, and controlled experiments. The implemented channels are:

- Channel 0: a sine-like ROM waveform.
- Channel 1: a signed ramp/sawtooth waveform.
- Channel 2: a square waveform.
- Channel 3: an alternating waveform useful for later correlation tests.

The AVG-ASP implements a causal moving average. For input sequence x[k], the ideal moving-average output for window length L is:

```
y[k] = (x[k] + x[k-1] + ... + x[k-L+1]) / L
```

Only power-of-two windows are used so that division can be implemented with an arithmetic right shift instead of a general hardware divider. The final implementation supports L = 1, 2, 4, 8, and 16 samples. The IRP-requested windows of 4, 8, and 16 samples are therefore directly available. The boundary rule is simple and deterministic: the AVG-ASP suppresses output until the selected window is full. For example, with L = 4, the first output appears after four accepted input data packets. This avoids ambiguous partial-window scaling at startup and after CLEAR.

The moving average is important because the reference-point algorithm is sensitive to random noise and quantization noise. A short window smooths the digitized signal before the later correlation and peak-detection stages. A larger window improves smoothing but increases latency and reduces response speed. In a real-time frequency relay this is a design trade-off: too much smoothing can delay detection, while too little smoothing can make the reference points less stable.

3. Packet Interface and NoC Use

The current ASPs are designed to sit behind a TDMA-MIN Network Interface (NI). The NI is considered part of the NoC fabric in the course brief, so the ASPs expose a compact node-side interface rather than implementing the whole NoC internally. Every command, data, event, and status item is represented as a 32-bit packet:

```
31..28  packet kind
27..24  command code or event code
23..20  destination node id
19..0   payload, sample, or result
```

Packet kinds:

```
0x1 command
0x2 data
0x3 status
0x4 event
```

Command codes:

```
0x0 NOP
0x1 CONFIG
0x2 START
0x3 STOP
0x4 CLEAR
```

The command path has `cmd_valid`, `cmd_ready`, and `cmd_word`. The output path has `out_valid`, `out_ready`, and `out_word`. AVG-ASP also has a streaming input path using `in_valid`, `in_ready`, and `in_word`. The design follows ready/valid semantics: a packet is transferred in a clock cycle when both valid and ready are high. If the downstream side is not ready, the ASP holds its output packet stable.

The ADC-ASP uses no stream input because it generates samples internally. The AVG-ASP consumes data packets from a previous source, normally ADC-ASP, and emits averaged data packets to the next ASP or to a destination NI. Status packets are continuously available as `status_word` for debug, SignalTap, or a later status read path.

4. ADC-ASP RTL Design

The ADC-ASP is an input/source ASP. Its job is to create one signed 16-bit sample at a programmable interval, wrap that sample into the common data-packet format, and send it to a configured destination node.

The CONFIG command payload is:

```
19..16  output destination node id
15..14  virtual channel select
13..10  sample divider
9..0    spare
```

The divider controls the sample rate relative to the ASP clock. If `sample_div = 0`, the block can emit a sample every available clock cycle when enabled and not backpressured. If `sample_div = n`, it emits one sample every `n + 1` available clock cycles. In a complete system, this divider would be chosen to match the desired sampling rate or a scaled test rate. The command sequence normally sent by ReCOP or Nios II is CONFIG followed by START. STOP disables sampling, and CLEAR resets phase, divider count, and any held output.

Internally, the ADC-ASP contains:

- Command decoder and configuration registers.
- Enable flag and simple control state.
- Divider counter.
- Phase counter.
- Virtual waveform generator.
- 16-bit sample register.
- Data packet builder.
- Status packet builder.

The waveform generator is combinational. Channel 0 uses a small sine-like lookup table. Channel 1 generates a signed ramp, currently used for the ADC-to-AVG verification because the expected average is easy to calculate by hand. Channel 2 generates a square wave, and channel 3 generates alternating signed values.

The data path is intentionally short: divider tick -> phase -> waveform generator -> sample register -> packet builder. Because there is no multiplication or memory block in this ASP, synthesis uses no DSP blocks and no block RAM. The current implementation uses a ROM constant for the sine-like values, which Quartus maps into logic because the table is very small.

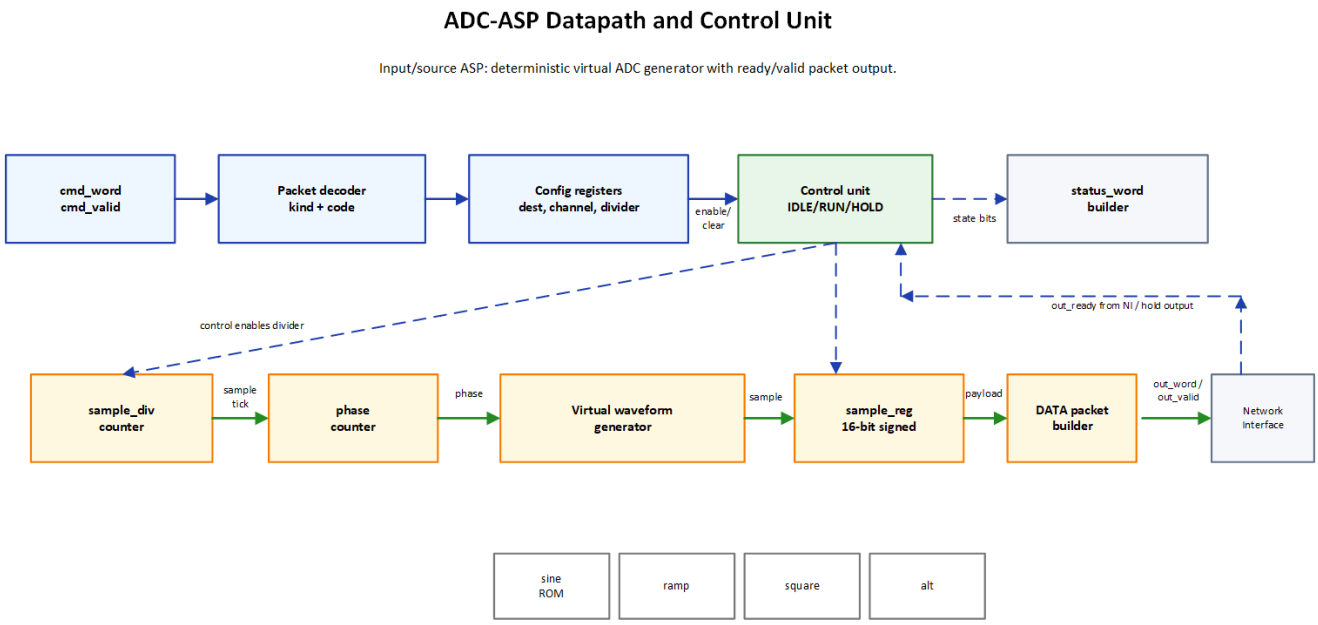

Figure 2. ADC-ASP datapath and control unit.

The status packet payload records the main debug state: enabled flag, selected channel, divider, and phase. This is useful while integrating with the TDMA-MIN interface, because a board-level debug path can check whether the ADC-ASP is configured and progressing.

5. AVG-ASP RTL Design

The AVG-ASP is a data-processing ASP. It consumes signed sample packets, maintains a sliding window, and emits signed average-result packets. The current implementation supports 1, 2, 4, 8, and 16 sample windows. The useful IRP configurations are therefore:

```
L = 4   log2(L) = 2
L = 8   log2(L) = 3
L = 16  log2(L) = 4
```

The CONFIG command payload is:

```
19..16  output destination node id
10..8   log2(window), where 0,1,2,3,4 mean 1,2,4,8,16 samples
7..0    spare
```

If a value above 4 is provided, the RTL clamps it to the 16-sample window. This avoids an illegal shift amount and gives deterministic behavior for invalid configuration values. START enables input acceptance, STOP disables input acceptance, and CLEAR resets the circular buffer, write index, valid count, running sum, average register, and held output flag.

The main AVG datapath is:

```
input data packet -> sample extractor -> 16 x 16-bit circular buffer
                  -> add incoming sample and subtract outgoing old sample
                  -> 20-bit running sum
                  -> arithmetic right shift by log2(window)
                  -> 16-bit avg_reg
                  -> output data packet
```

The running-sum method avoids re-summing the entire window for every output. When a new sample is accepted, the oldest sample in the active window is subtracted and the new sample is added. Once the window is full, this produces one new average per accepted input sample. The arithmetic right shift preserves signed values and implements division by the power-of-two window length.

The 20-bit running sum is sufficient for a 16-sample window of signed 16-bit values. The maximum positive sum is `16 * 32767 = 524272`, and the most negative sum is `16 * -32768 = -524288`, which fits the signed 20-bit range. The output is resized back to a signed 16-bit sample before packet construction.

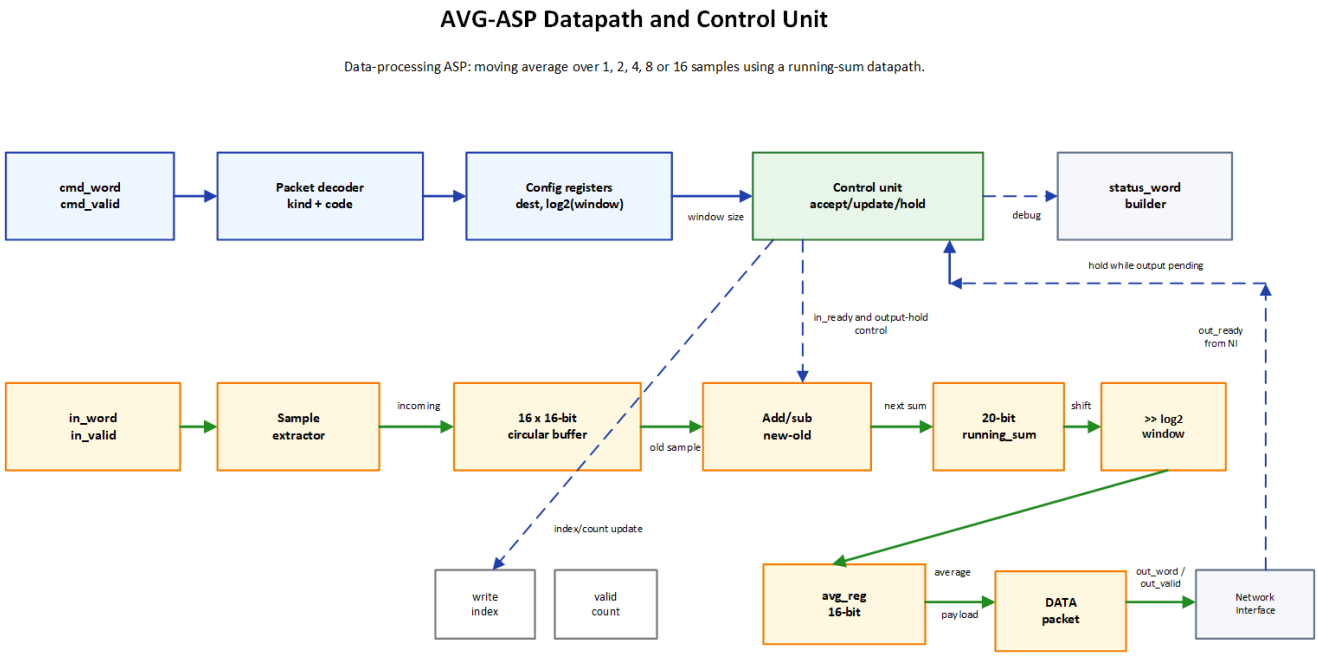

Figure 3. AVG-ASP datapath and control unit.

The AVG control unit also handles backpressure. `in_ready` is asserted only when the ASP is enabled and not already holding an output packet. This keeps the one-cycle datapath simple and prevents a new sample from overwriting state while an old output is waiting for acceptance. If the next ASP or NI is ready, the output packet is accepted and the AVG-ASP can receive another input sample on the following cycle.

6. Verification Results

The current code was checked with ModelSim Intel FPGA Starter Edition 10.5b.

First, the focused ADC-to-AVG test compiled `adc_asp`, `avg_asp`, and `tb_adc_avg_pipeline`. It configured AVG for a 4-sample window and configured ADC channel 1 as a ramp source. The first ADC samples were:

```
-2048, -1984, -1920, -1856
```

The first AVG output was:

```
(-2048 + -1984 + -1920 + -1856) / 4 = -1952
```

The transcript confirmed `AVG sample = -1952` at 430 ns, followed by the expected ramping average values. This proves that the packet connection, signed sample extraction, circular-buffer update, running sum, and output packet generation work together for the ADC/AVG pipeline.

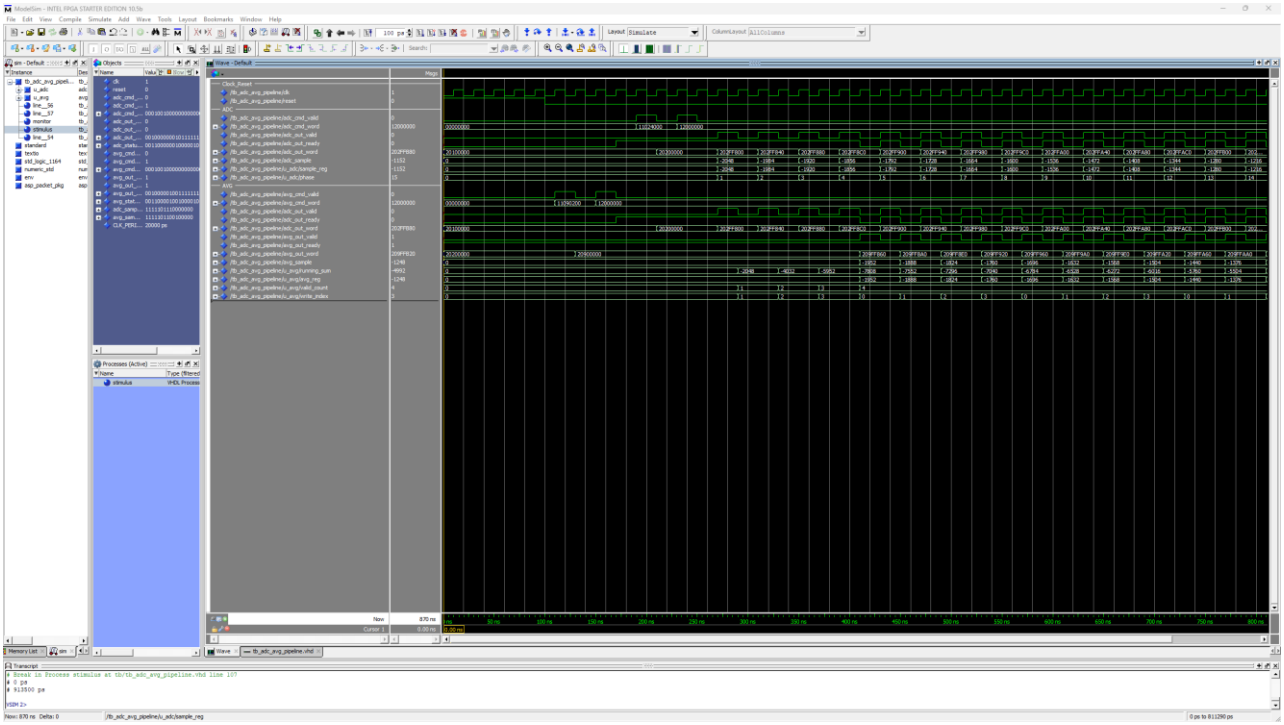

Figure 4. ModelSim waveform for the ADC-ASP to AVG-ASP pipeline.

Figure 4 shows the same ADC-to-AVG simulation in the ModelSim waveform window. The ADC ramp samples are visible on `adc_sample`, while `avg_sample`, `running_sum`, `valid_count`, and `write_index` show the AVG-ASP filling the four-sample window and then producing the expected moving-average outputs.

Second,a new focused AVG window-16 test was added in `tb/tb_avg_window16.vhd` and run with `run_avg_window16_wave.do`. It configured `log2(window)=4`, sent samples 1 through 16, and asserted that the first output is `floor(136/16) = 8`. The simulation completed with:

```
AVG window-16 test passed
Errors: 0, Warnings: 0
```

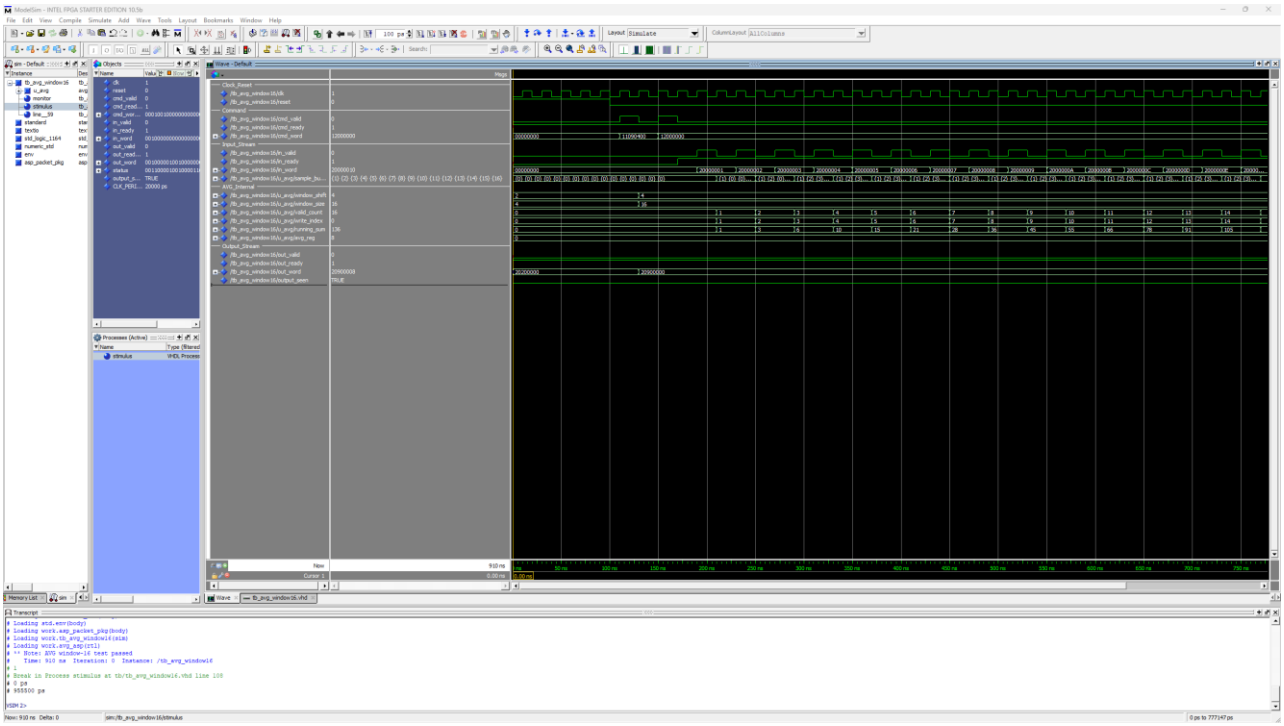

Figure 5. ModelSim waveform for the AVG-ASP 16-sample moving-average test.

Figure 5 verifies the 16-sample AVG configuration independently from the ADC source. The testbench sends the known input sequence 1, 2, 3, ..., 16 directly into AVG-ASP. These are deliberate verification samples, not random data. The waveform shows `valid_count` filling to 16, `running_sum` accumulating to 136, and the first output average becoming 8 after the right-shift divide by 16. This confirms that the implemented AVG-ASP supports the IRP-required 16-sample window.

These tests do not replace full NoC integration testing, but they do verify the behaviour of the two individual ASPs and the local ready/valid packet interface.

7. Conclusion

The implemented ADC-ASP and AVG-ASP provide the front end of the group frequency-measurement ASP pipeline. ADC-ASP generates deterministic virtual samples suitable for testbeds and later NoC integration. AVG-ASP implements the required moving-average filtering behaviour using a compact running-sum datapath, supports the IRP-relevant 4, 8, and 16 sample windows, and uses packet-level control so that ReCOP or Nios II can configure it without direct internal access.

Simulation shows that both ASPs compile and operate correctly in the task pipeline and in the focused ADC/AVG waveform test. A new window-16 AVG test also passes. Quartus synthesis confirms that both blocks are small and comfortably meet a 50 MHz target clock on Cyclone V. The next integration step is to connect these ASPs to the selected TDMA-MIN Network Interface and verify the same command/data packets through the actual NoC fabric.

References

[1] Z. Salcic and M. Biglari-Abhari, "COMPSYS701 Advanced Digital Design, 2026 Individual Project", University of Auckland, 2026.

[2] Z. Salcic and M. Biglari-Abhari, "COMPSYS 701 Advanced Digital Design, Semester 1 2026: Heterogeneous Multiprocessor System on Chip for Mixed-criticality Applications", University of Auckland, 2026.

[3] Z. Salcic and R. Mikhael, "A new method for instantaneous power system frequency measurement using reference points detection", Electric Power Systems Research, vol. 55, no. 2, pp. 97-102, 2000.

[4] "Frequency relay - Power Signal Model and Power Signal Generator", COMPSYS 701 individual-project supporting note, 2026.

[5] "Frequency relay VHDL design", COMPSYS 701 additional frequency-analysis notes, 2026.